

LISTA PROTOCOLO DE RESILIÊNCIA

(Tratamento de Exceções)

A Sua Missão: Você foi contratado pela Nexus Tech, uma startup de games. O jogo de vocês está cheio de bugs e fechando sozinho no computador dos jogadores. Sua missão é usar o Tratamento de Exceções do Java para "capturar" esses erros e não deixar o jogo crashar (fechar do nada).

SETUP: A FORJA DA IDE

(Siga o passo a passo exato. Se você não fizer isso, o VS Code vai dar o erro vermelho "Incorrect Package".)

1. Aperte **F1** (ou Ctrl + Shift + P) no VS Code.
2. Digite **Java: Create Java Project** e aperte Enter.
3. Escolha **No build tools**, selecione a Área de Trabalho e chame o projeto de **missao-07-excecoes**.
4. Delete o arquivo **App.java** que ele criar.
5. **Criando a estrutura de pastas correta:**
 - Não tente criar arquivos "soltos".
 - No explorador de arquivos (lado esquerdo), clique com o botão direito em **src** e clique em **New Java Package**.
 - Digite exatamente assim usando barras: **br/com/nexustech/main** e dê Enter. (O VS Code vai montar a árvore de pastas correta).
 - Agora, clique com o botão direito nessa nova pasta **main** e escolha **New Java File**. Chame de **Main.java**. A IDE fará o resto!

NÍVEL 1: O TRAJE ANTIBOMBAS (Unchecked Exceptions)

DICA DE SINTAXE: O Bloco Try-Catch

Quando achar que um código vai dar erro, "abraçe" ele com esta estrutura:

```
try {  
    // Código perigoso aqui  
} catch (NomeDoErro e) {  
    // O que o sistema deve fazer para se salvar se o erro acontecer  
}
```

Exercício 1: O Bug do K/D (Divisão por Zero)

Na sua classe `Main` (dentro do `public static void main`), vamos simular um bug matemático.

- ☐ Crie duas variáveis: `int kills = 15;` e `int deaths = 0;`
- ☐ Peça para o Java imprimir a divisão: `System.out.println(kills / deaths);`

Missão: Rode o código. O jogo vai crashar e soltar letras vermelhas no console. Leia as letras vermelhas, encontre o nome da Exceção (termina com Exception) e anote no seu caderno/código.

Exercício 2: Consertando o K/D

O jogo não pode fechar na cara do usuário só porque ele não morreu!

- ☐ Apague a linha do `System.out.println` do Exercício 1 e recrie ela dentro de um bloco try.
- ☐ No catch, coloque o nome do erro que você descobriu no Ex. 1.
- ☐ Dentro do catch, imprima a mensagem: "Taxa K/D: Jogador Invicto!". (Rode e veja que o erro sumiu!).

Exercício 3: O Inventário Bugado

O inventário do nosso RPG só tem 3 espaços (índices 0, 1 e 2).

- ☐ Crie um array: `String[] inventario = new String[3];`
- ☐ Crie um bloco try-catch.
- ☐ No try, tente forçar a "Espada" no espaço 5: `inventario[5] = "Espada";`
- ☐ No catch, capture a `ArrayIndexOutOfBoundsException` e imprima: "Inventário cheio!".

Exercício 4: O Fantasma (O Mito do NullPointerException)

- ☐ Crie uma variável nula: `String jogador = null;`

Regra do Chefe: É proibido usar try-catch para arrumar NullPointerException. Isso é código amador.

Missão: Use um simples if/else (Programação Defensiva) para checar: Se o jogador for diferente de `null`, imprima o nome dele. Senão, imprima "Jogador desconectado".

NÍVEL 2: O CONTRATO OBRIGATÓRIO (Checked Exceptions)

DICA DE SINTAXE: Arremessando e Avisando

`throw` : Joga a bomba dentro do método. (`throw new Exception("Erro!");`)

`throws` : Avisa o perigo na assinatura do método. (`public void testar() throws Exception { }`)

Exercício 5: Simulando a Queda de Internet

Vamos criar um método novo. Lembre-se: no Java, um método não pode ser criado dentro de outro!

- ☐ Localize a última chave `}` do seu método `public static void main(String[] args)`.
- ☐ Pule uma linha **abaixo** dessa chave (mas ainda antes da última chave final da classe).
- ☐ Crie o método estático ali: `public static void conectarServidor()`
- ☐ Coloque a palavra `throws Exception` logo após os parênteses `()`.
- ☐ Dentro das chaves deste novo método, force a queda com o arremesso físico: `throw new Exception("Servidor caiu!");`

(Dica de Check: O seu código deve ficar parecido com isto:)

```
public class Main {
    public static void main(String[] args) {
        // ... (código do main) ...
    } // Fim do main

    // SEU NOVO MÉTODO COMEÇA AQUI:
    public static void conectarServidor() throws Exception {
        throw new Exception("Servidor caiu!");
    }
}
```

Exercício 6: A Assinatura do Contrato

- ☐ Volte para dentro do seu `public static void main`.
- ☐ Chame o seu método: `conectarServidor();`

Missão: Note que a IDE vai reclamar. O Java te OBRIGA a tratar! Envolve essa chamada num try-catch (capturando `Exception e`). No catch, imprima `e.getMessage()`.

DICA DE SINTAXE: O bloco Finally

```
catch (Exception e) {  
}  
finally {  
    // Roda SEMPRE, dando erro ou não!  
}
```

Exercício 7: O Zelador (Fechando portas)

- ☐ Logo abaixo do catch do Exercício 6, adicione o bloco finally.
- ☐ Imprima: "Fechando portas de rede do jogo...". Rode o código e veja ele ser executado por último.

NÍVEL 3: DOMINANDO AS REGRAS (Exceções Customizadas)

DICA DE SINTAXE: Criando sua própria Exceção

```
public class MinhaExcecao extends RuntimeException {  
    public MinhaExcecao() {  
        super("Minha mensagem de erro personalizada!");  
    }  
}
```

Exercício 8: Forjando a Regra de Negócio

O Java não sabe o que é um RPG. Vamos ensinar criando uma classe nova!

- ☐ Crie a classe `NivelInsuficienteException` no pacote `br.com.nexustech.exception`.
- ☐ Faça ela aplicar herança (extends) de `RuntimeException`.
- ☐ Crie o construtor dela e use o `super("...");` para mandar a mensagem: "Seu nível é muito baixo para esta masmorra!".

Exercício 9: A Masmorra Fechada

- ☐ Crie a classe `Masmorra` (no pacote `.model`).
- ☐ Crie nela o método `public void entrar(int nivelJogador)`.
- ☐ Se o nível for menor que 50, lance a exceção que você acabou de criar no Ex. 8 (`throw new NivelInsuficienteException();`).
- ☐ Vá na classe Main, instancie a Masmorra e tente entrar com um jogador nível 20. Lembre-se de usar try-catch para o jogo não fechar sozinho!

NÍVEL BOSS: O ARQUITETO MATCHMAKER

Agora vamos juntar a Aula 4 (Polimorfismo/Interfaces) com a Aula 7 (Exceções)! Siga os passos com calma:

Exercício 10: A Interface

- ☐ Crie a interface `ModoJogo` com um método vazio `void buscarPartida();`

Exercício 11: As Opções

- ☐ Crie as classes `ModoCasual` e `ModoRanqueado`. Faça elas assinarem o contrato (`implements ModoJogo`) e coloque um `println` diferente no método de cada uma.

Exercício 12: A Exceção Suprema

- ☐ Crie a classe `BanidoException` (Herde de `Exception` normal, não Runtime). Mensagem: "Jogador Banido!".

Exercício 13: O Sistema

- ☐ Crie a classe `Matchmaker`.
- ☐ Crie o método: `public void encontrarSala(ModoJogo modo, boolean jogadorBanido) throws BanidoException`
- ☐ Lógica: Se for banido, dispare a `BanidoException`. Se não for, execute o `modo.buscarPartida();`.

Exercício 14: A Batalha Final

- ☐ Na sua `Main`, instancie o Matchmaker, o modo Casual e o Ranqueado. Use um try-catch para tentar buscar uma sala para um jogador banido. O sistema deve capturar o banimento com elegância!

Terminou? Parabéns! Seu jogo está blindado contra falhas! Faça o git commit e mande para a nuvem.

DESAFIO EXTRA: O COFRE DO GABARITO (Engenharia e Segurança)

Onde estão as respostas?

O gabarito oficial desta lista foi encriptado. Para provar que você dominou o controle de fluxo e o tratamento de exceções (capturando falhas de padding criptográfico), você precisará fazer um ataque de Força Bruta para destravar o link do Drive.

- **Link Cifrado:**

U2FsdGVkX1/Jz86x4/Ydu3FZFw5pSo86xHG1MwpCFX/
Dnn9uCMDd3xLNn61XZouvQy6G2FIhyQXAQwvTwn3/01JGIoIh5RN4NXgs+kdpcf6afHmSMvCZ
u0EiiiXlVpB2EQGIKDLIAU9c1aQx6bzEgQ==

- **Algoritmo:** aes-256-cbc com pbkdf2 e -iter 1000
- **Padrão da Senha:** Exatamente 5 caracteres minúsculos/números.
- **Dica de Inteligência (Vazamento):** A senha começa com jav

Sua Missão: Complete os espaços vazios do código abaixo, rode e pegue o link!

```
package br.com.frenteira.main;

import javax.crypto.Cipher;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Base64;

public class App {
    public static void main(String[] args) {
        String encryptedB64 = "U2FsdGVkX1/Jz86x4/Ydu3FZFw5pSo86xHG1MwpCFX/  
Dnn9uCMDd3xLNn61XZouvQy6G2FIhyQXAQwvTwn3/01JGIoIh5RN4NXgs+kdpcf6afHmSMvCZu0EiiiXlVpB2EQGIKDLIAU9c1aQx  
6bzEgQ==";
        // Limpa possíveis quebras de linha do Base64
        encryptedB64 = encryptedB64.replaceAll("\s", "");

        // 1. Decodifica o Base64 para Bytes
        // 2. Extrai o Salt (do byte 8 ao 15) e o Texto Cifrado (do 16 em diante)
        // O padrão da senha é "javXX" contendo letras e números
        String charset = "abcdefghijklmnopqrstuvwxyz0123456789";
        System.out.println("Iniciando ataque de força bruta no link da NexusTech...");
        long startTime = System.currentTimeMillis();

        // 3. Força Bruta: Combinando os dois últimos caracteres
        for (char c1 : charset.toCharArray()) {
            for (char c2 : charset.toCharArray()) {
                String testPass = "jav" + c1 + c2;
                try {
                    // PBKDF2 gera Key e IV juntos. 32 bytes (Key) + 16 bytes (IV) = 48 bytes
                    byte[] key = Arrays.copyOfRange(keyAndIv, 0, 32);
                    byte[] iv = Arrays.copyOfRange(keyAndIv, 32, 48);

                    // Prepara o motor do AES-256-CBC
                    // Tenta descriptografar. Se a chave estiver errada, o padding falha e lança
```

Exception

```
String result = ""; // Lógica real de descriptografia deve preencher aqui

// 4. Checa se revelou o link com sucesso
if (result.contains("http")) {
    long endTime = System.currentTimeMillis();
    System.out.println("\n SUCESSO! A criptografia foi quebrada!");
    System.out.println("Senha encontrada: " + testPass);
    System.out.println("Link revelado: " + result.trim());
    System.out.println("Tempo de execução: " + (endTime - startTime) + "ms");
    return; // Interrompe a execução após achar a senha
}
} catch (Exception e) {
    // Uma senha incorreta vai acionar a BadPaddingException.
    // Nós simplesmente ignoramos o erro e o loop continua testando a próxima!
}
}
}
System.out.println("\nForça bruta concluída. Senha não encontrada.");
}
}
```